```
        if (random.random() < epsilon):
            action_ = np.random.randint(0,4)
        else:
            action_ = np.argmax(qval_)

        action = action_set[action_]
        game.makeMove(action)
        state2_ = game.board.render_np().reshape(1,64) +
np.random.rand(1,64)/100.0
        state2 = torch.from_numpy(state2_).float()
        reward = game.reward()
        done = True if reward > 0 else False
        exp =  (state1, action_, reward, state2, done)
        replay.append(exp)
        state1 = state2

        if len(replay) > batch_size:
            minibatch = random.sample(replay, batch_size)
            state1_batch = torch.cat([s1 for (s1,a,r,s2,d) in
minibatch])
            action_batch = torch.Tensor([a for (s1,a,r,s2,d) in
minibatch])
            reward_batch = torch.Tensor([r for (s1,a,r,s2,d) in
minibatch])
            state2_batch = torch.cat([s2 for (s1,a,r,s2,d) in
minibatch])
            done_batch = torch.Tensor([d for (s1,a,r,s2,d) in
minibatch])
            Q1 = model(state1_batch)
            with torch.no_grad():
                Q2 = model2(state2_batch)        2
            Y = reward_batch + gamma * ((1-done_batch) * \
            torch.max(Q2,dim=1)[0])
            X = Q1.gather(dim=1,index=action_batch.long() \
            .unsqueeze(dim=1)).squeeze()
            loss = loss_fn(X, Y.detach())
            print(i, loss.item())
            clear_output(wait=True)
            optimizer.zero_grad()
            loss.backward()
            losses.append(loss.item())
            optimizer.step()

            if j % sync_freq == 0:               3
                model2.load_state_dict(model.state_dict())
        if reward != -1 or mov > max_moves:
            status = 0
            mov = 0

losses = np.array(losses)
```